

ALT FUN - SECURITY REVIEW

Alt Fun Review.

A security review of the Alt Fun bonding, routing, launch, fee, and graduation flows.

CLIENT

Alt Fun

AUDITORS

Jakub Zmyslowski -
Osman Ozdemir - Ali
Shehab - Wafflemakr -
Michael Lett

AUDIT WINDOW

May 19 - May 28, 2026

FINAL REPORT

July 1, 2026

§ 00

Contents.

46 pages — overview, scope, 36 findings, disclaimer, and about Guardian.

§ 01	Overview	p. 03
§ 02	Scope & Classifications	p. 04
	Files in Scope	p. 05
§ 03	Findings	p. 06
	R1-C-01 to R3-M-02	p. 9-43
	Low • R1-L-01 to R1-L-10	p. 11-20
	Info • R1-I-01 to R3-I-01	p. 21-44
§ 04	Disclaimer	p. 45
§ 05	About Guardian	p. 46

§ 01

Overview.

Guardian reviewed Alt Fun over one main review and two remediation-review rounds. The final report includes one Critical, one High, three Medium, ten Low, and twenty-one Informational findings.

AUDIT FIRM	Guardian	REPOSITORY	alt-fun
CLIENT	Alt Fun	METHODOLOGY	Static - Manual - Test - Fuzz
PREPARED BY	Zmyslowski - Ozdemir - Shehab - Wafflemakr - Lett	MAIN REVIEW COMMIT	625fd21454a5232f2c8582d4b1135a9bdeafedec
AUDIT WINDOW	May 19 - May 28, 2026	REMEDIATION COMMIT	2ee8c0faf8363d21af5870f3c6d1003602d8390e
FINAL REPORT	July 1, 2026		

1.1 Confidence Ranking

4/5

High Confidence

Guardian assigns a Confidence Ranking of 4 out of 5. The review covered the core launch and graduation flows across one main review and two remediation rounds.

1 V. LOW	2 LOW	3 MOD	4 HIGH	5 V. HIGH
-------------	----------	----------	-----------	--------------

1.2 Findings at a Glance

SEVERITY	TOTAL	PENDING	DECLINED	ACK	PARTIAL	RESOLVED
Critical	1	0	0	0	0	1
High	1	0	0	0	0	1
Medium	3	0	0	1	0	2
Low	10	0	0	2	0	8
Info	21	0	0	3	0	18

§ 02

Scope & Classifications.

The audit covered Alt Fun smart contracts in `packages/contracts/src`, including Bonding, routing, factory, fee, LP-lock, pair, token, and Zap flows. Findings are grouped by review round and classified by worst-case impact and likelihood.

2.1 Severity Matrix

	IMP · HIGH	IMP · MEDIUM	IMP · LOW	IMPACT · HIGH	Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.
LIK · HIGH	Critical	High	Medium	IMPACT · MEDIUM	Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt.
LIK · MED	High	Medium	Low	IMPACT · LOW	Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.
LIK · LOW	Medium	Low	Low	LIKELIHOOD · HIGH	Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.
				LIKELIHOOD · MEDIUM	Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.
				LIKELIHOOD · LOW	Unlikely to ever occur in production.

§ 02

Scope.

Files in scope, grouped by repository. SLOC counts are physical source lines.

alt-fun

8 CONTRACTS · 1,355 SLOC

packages/contracts/src/		
Bonding.sol		604
Factory.sol		44
FeeVault.sol		146
LPlock.sol		62
Pair.sol		71
Router.sol		104
Token.sol		22
Zap.sol		302

§ 03

Findings.

Thirty-six findings across three review rounds: one Critical, one High, three Medium, ten Low, and twenty-one Informational. Status reflects the latest mapped Guardian status: In Fix Review is shown as Resolved; Acknowledged remains Acknowledged.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
R1-C-01	Dust Pre-Seed Allows Full Liquidity Drain	Logical Error	CRITICAL	RESOLVED	p. 9
R1-H-01	Tiny LP seed skews graduation price	Logical Error	HIGH	RESOLVED	p. 10
R1-L-01	Graduation not locked during sells	Unexpected Behavior	LOW	RESOLVED	p. 11
R1-L-02	Low LT price can block graduation	Unexpected Behavior	LOW	RESOLVED	p. 12
R1-L-03	Accrued LT fees skew exchange rate	Trust Assumptions	LOW	RESOLVED	p. 13
R1-L-04	LT redeploy freezes exchangeRate	Logical Error	LOW	RESOLVED	p. 14
R1-L-05	Stale getAmountOut quotes for graduated tokens	Logical Error	LOW	ACKNOWLEDGED	p. 15
R1-L-06	Events can emit misleading amounts	Unexpected Behavior	LOW	RESOLVED	p. 16
R1-L-07	'MIN_USDC_AMOUNT' can desync from LT floor	Configuration	LOW	RESOLVED	p. 17
R1-L-08	Graduation event overstates LP tokens	Unexpected Behavior	LOW	RESOLVED	p. 18
R1-L-09	Floor-bump buys strand LT dust	Logical Error	LOW	ACKNOWLEDGED	p. 19
R1-L-10	'getAmountOut' overstates buys near sellout	Logical Error	LOW	RESOLVED	p. 20

§ 03

Findings, continued.

Index page 2 of 3.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
R1-I-01	Inconsistent `balanceOf` vs `tokenBalance()`	Best Practices	INFO	RESOLVED	p. 21
R1-I-02	`Usd` constants are actually USDC-denominated	Documentation	INFO	RESOLVED	p. 22
R1-I-03	Seed Fee Reduces Curve Floor	Validation	INFO	RESOLVED	p. 23
R1-I-04	Dust Fee Rounding Under-accrues	Rounding	INFO	RESOLVED	p. 24
R1-I-05	Zero LP Lock Can Be Rewritten	Validation	INFO	RESOLVED	p. 25
R1-I-06	Trader-Favorable Rounding in Curve Quote Math	Rounding	INFO	ACKNOWLEDGED	p. 26
R1-I-07	Zap stores an unused UniswapV2 router	Best Practices	INFO	RESOLVED	p. 27
R1-I-08	Hardcoded 0.3% V2 fee desyncs from HyperSwap	Logical Error	INFO	RESOLVED	p. 28
R1-I-09	Documentation drift	Documentation	INFO	RESOLVED	p. 29
R1-I-10	Seed Buy Can Be Unwound	Documentation	INFO	RESOLVED	p. 30
R2-M-01	Tiny Minted Preseeds Still Cause Graduation Skew	Logical Error	MEDIUM	RESOLVED	p. 31
R2-I-01	Unused Zap import	Superfluous Code	INFO	RESOLVED	p. 32

§ 03

Findings, continued.

Index page 3 of 3.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
R2-I-02	Zap Bonding rotation lacks runbook	Documentation	INFO	RESOLVED	p. 33
R2-I-03	Stale 'MIN_SEED_USDC' Floor Docs	Documentation	INFO	RESOLVED	p. 34
R2-I-04	GlobalStorage rotation affects trades	Documentation	INFO	RESOLVED	p. 35
R2-I-05	'Router.initialize' missing factory zero-check	Informational	INFO	RESOLVED	p. 36
R2-I-06	FeeVault rotation fragments claims	Documentation	INFO	RESOLVED	p. 37
R2-I-07	Renounced ownership can break admin paths	Informational	INFO	ACKNOWLEDGED	p. 38
R2-I-08	Single-step creator transfer	Informational	INFO	ACKNOWLEDGED	p. 39
R2-I-09	Factory router can be frozen wrong	Validation	INFO	RESOLVED	p. 40
R2-I-10	'FeeVault.setFeeTo' redirects accrued fees	Best Practices	INFO	RESOLVED	p. 41
R3-M-01	Pre-Seed LP Subsidy via Protocol LT Correction	Logical Error	MEDIUM	RESOLVED	p. 42
R3-M-02	Hostile Preseed Can Consume Swap Budget	Logical Error	MEDIUM	ACKNOWLEDGED	p. 43
R3-I-01	Factory ↔ Router Circular Import	Best Practices	INFO	RESOLVED	p. 44

CRITICAL R1-C-01

RESOLVED

Dust Pre-Seed Allows Full Liquidity Drain.

CATEGORY Logical Error

LOCATION src/Bonding.sol:1226-1231

REPO Round 1 - Main Review

DESCRIPTION

`Bonding.finalizeGraduation` assumes the post-graduation Uniswap V2 pair can always be either seeded directly or repaired through the rebalance path in `_seedUniswapV2Direct`. However, that assumption is false when the pair already exists in a dust-synced state with nonzero reserves but zero LP supply. Since anyone can create the `(launchedToken, LT)` pair and both token addresses are known in advance, an attacker can pre-create the pair, transfer 1 wei of each asset into it, and call `sync`, causing the pair to end up with reserves `(1,1)` while `totalSupply == 0`.

From that point onward, `_seedRebalancing` does not meaningfully correct the pool because `_pairRebalance` computes `expectedOut == 0` against such tiny reserves and returns without executing a swap, causing the code to fall through to `_routerDepositAndDispose`, which results in liquidity being added at the attacker-controlled 1:1 ratio rather than the intended cached graduation ratio.

Attack Steps:

- Attacker creates the `(launchedToken, LT)` pair
- Transfers 1 wei of each token directly to the pair and calls `sync`
- This updates the reserves and sets the ratio to 1:1
- `finalizeGraduation` is called and `_routerDepositAndDispose` adds liquidity at 1:1 ratio rather than the actual curve ratio.
- Attacker sells previously acquired launched tokens at artificially manipulated 1:1 ratio

As a result, the graduated pool opens at an artificial 1:1 ratio instead of the curve-close ratio, allowing previously accumulated launched tokens to be sold into a severely mispriced market and enabling an attacker to drain almost all LT liquidity immediately after graduation.

RECOMMENDATION

Treat pairs with `totalSupply() == 0` as pristine regardless of synced dust reserves. Additionally, consider requiring the post-rebalance reserve ratio to be validated against the cached curve-close ratio before adding liquidity.

RESOLUTION

Alt Fun Team - Resolved in `alt-fun#1137`.

HIGH **R1-H-01**

RESOLVED

Tiny LP seed skews graduation price.

CATEGORY Logical Error

LOCATION src/Bonding.sol:1293-1306

REPO Round 1 - Main Review

DESCRIPTION

`finalizeGraduation` assumes that a pre-existing V2 pair with nonzero reserves can be safely repaired by `_seedRebalancing` before liquidity is deposited. This is false when an attacker creates the pair and mints a tiny amount of LP at a hostile ratio. The attacker can seed the pair with amounts just above Hyperswap V2 minimum liquidity requirements. This gives them only dust LP, but records the attacker-controlled reserves in the pair. During graduation, `_seedUniswapV2Direct` sees nonzero reserves and enters `_seedRebalancing`. The rebalance/deposit flow then adds liquidity against the manipulated reserve ratio instead of guaranteeing the cached curve-close ratio. The router only pulls the ratio-matched subset, and leftover launched tokens are burned. As a result, the graduated pool can open at an incorrect price with substantially less liquidity than intended.

RECOMMENDATION

Update `_pairRebalance` so it does not skip the rebalance when the computed swap output rounds down to zero. In that case, calculate the minimum input required to receive 1 wei of output and execute that swap if it fits within the rebalance budget. This prevents dust LP preseeds from bypassing the swap-based repair path due to integer rounding. For cases where a swap is impossible, handle dust-only preseeds with a tightly bounded `donate + sync + mint` fallback so finalization can still progress without opening the pool at the attacker-controlled ratio.

RESOLUTION

Alt Fun Team - Resolved in `alt-fun#1195`.

LOW **R1-L-01**

RESOLVED

Graduation not locked during sells.

CATEGORY Unexpected Behavior

LOCATION `src/Bonding.sol:619-634`

REPO Round 1 - Main Review

DESCRIPTION

A token can become graduable without a trade when the LT perp value increases, as graduation uses the current LT exchange rate and the curve's stored LT reserve. However, this state is not locked in through the sell path as it is through the buy path. The sell path does not check `canGraduate` before allowing a sale to execute. A token holder can therefore sell after the token has become graduable but before anyone calls `triggerGraduation`, which can push the token back below the graduation threshold. Consequently, graduation becomes race-dependent, and a token that was already eligible to graduate can miss that opportunity to graduate if a sell lands first.

RECOMMENDATION

Check `canGraduate` before executing sells, and reject curve sells once the token is already graduable. Alternatively, document this behavior as intended.

RESOLUTION

Alt Fun Team - Resolved in `alt-fun#1191`.

LOW **R1-L-02**

RESOLVED

Low LT price can block graduation.

CATEGORY Unexpected Behavior

LOCATION contracts/UniswapV2Pair.sol:74

REPO Round 1 - Main Review

DESCRIPTION

During graduation, the protocol withdraws the real LT raised by the curve and later deposits it into the HyperSwap V2 pair. The internal bonding curve stores reserves as `uint256`, but the final HyperSwap V2 pair stores reserves as `uint112`. This creates a mismatch when the LT exchange rate becomes extremely small.

This can become problematic if the supported leverage LT suffers a severe price decline or liquidation during the curve phase. Even if the LT exchange rate does not fall to zero because some assets remain idle on HyperEVM, the exchange rate can become low enough that the same graduation value is represented by a very large number of LT units. The bonding curve can continue accounting for those large LT-unit reserves, but the final V2 pair cannot store reserves above `uint112.max`. If `ltFromPair` exceeds that limit, phase 2 finalization can revert, leaving the token stuck in `Graduating` state after curve trading has already been frozen.

RECOMMENDATION

Be aware that extremely low LT exchange rates can prevent HyperSwap V2 finalization, document this limitation clearly, and avoid launching pairs near this reserve limit. If an LT is liquidated or impaired, consider pausing mints or alternatively making `canGraduate` refuse entering graduation until finalization can safely fit within the V2 `uint112` reserve limit.

RESOLUTION

Alt Fun Team - Resolved in `alt-fun#1189`.

LOW **R1-L-03**

RESOLVED

Accrued LT fees skew exchange rate.

CATEGORY Trust Assumptions

LOCATION src/LeveragedToken.sol:178

REPO Round 1 - Main Review

DESCRIPTION

Alt Fun relies on the BounceTech LT `exchangeRate` for launch pricing and graduation checks. However, this is a view function and does not apply accrued streaming fees. Those fees are only applied when `_checkpoint` runs during state changing LT operations such as mint or redeem. During launch, Alt Fun computes the curve's virtual LT reserve from `exchangeRate`, then the mandatory seed buy calls `mint`, which will checkpoint the LT and reduce the rate immediately after the curve is initialized. The same issue applies to graduation. `canGraduate` can return true using a pre-checkpoint rate, allowing the token to enter `Graduating` phase even if the post-checkpoint reserve value would be below the threshold. As a result, launch pricing and graduation eligibility can happen based on overstated LT values.

RECOMMENDATION

Document that launch pricing and graduation checks may use a stale, pre-checkpoint LT exchange rate. Monitor `lastCheckpoint` for every supported LT, and warn users in the frontend when `block.timestamp - lastCheckpoint` exceeds a safe threshold. Consider exposing a fee-adjusted exchange rate view function.

RESOLUTION

Alt Fun Team - Resolved in `alt-fun#1188`.

LOW R1-L-04

RESOLVED

LT redeploy freezes exchangeRate.

CATEGORY Logical Error

LOCATION src/Bonding.sol:124

REPO Round 1 - Main Review

DESCRIPTION

`Factory.redeployLt(oldLT)` retires an LT and spins up a new one at a different address. The old LT contract isn't destroyed, just removed from the registry (`ltExists[oldLT] = false`). `mint`, `redeem`, and `exchangeRate` on the LT don't check the registry, so the old LT still works after redeploy. The problem is `exchangeRate` goes flat. After redeploy, keepers stop opening Hypercore positions on the old LT, so `hyperLiquidValue` and `spotAssetValue` stay at 0. Only `baseAssetBalance - debt` is left in `totalValue`.

So `exchangeRate` is stuck at whatever it was at redeploy time and doesn't track the underlying perp anymore. Tokens referencing the old LT keep trading but the LT side becomes a stable vault. The leveraged part of the product silently disappears. `canGraduate` uses `realLtRaised * exchangeRate ≥ graduationThresholdUsd`. With `exchangeRate` frozen, the USD trigger only fires by raising more LT.

Tokens counting on LT appreciation to graduate just sit on the curve.

The precondition here is that `Factory.redeployLt` reverts with `StillHasMargin` unless the operator first closes the old LT's margin position, so this only fires if redeployment happens while Alt Fun curves still reference the old LT; i.e. the operator empties the margin but Alt Fun tokens have not yet unwound their LT holdings.

RECOMMENDATION

Consider clearly documenting this behavior so that users are fully aware of such mechanism.

RESOLUTION

Alt Fun Team - Resolved in `alt-fun#1193`.

LOW R1-L-05

ACKNOWLEDGED

Stale getAmountOut quotes for graduated tokens.

CATEGORY Logical Error

LOCATION src/Router.sol:61

REPO Round 1 - Main Review

DESCRIPTION

`Router.graduate()` pulls LT out of the curve pair with `pair.transferAsset()` but never calls `pair.swap()` to update `_pool.assetReserve`. So after phase 1 the pair's actual LT balance is 0 but `pair.getReserves()` still reports the pre-graduation value (`virtualLtReserve + ltFromPair`).

TEXT

```
1 function graduate(address token, uint256 amount) external onlyRole(BONDING_ROLE) {
2     address asset = assetTokenFor(token);
3     address pairAddr = factory.getPair(token, asset);
4     if (pairAddr == address(0)) revert PairNotFound();
5     IPair(pairAddr).transferAsset(msg.sender, amount);
6 }
```

`Router.getAmountOut` and `Router.previewBuy` read `pair.getReserves()` and don't check lifecycle:

TEXT

```
1 (uint256 reserveToken, uint256 reserveAsset) = pair.getReserves();
2 uint256 k = pair.k();
```

For graduated tokens these return prices on inflated reserves. The pair has no LT left to back any quote.

RECOMMENDATION

Gate the view functions on lifecycle:

TEXT

```
1 function getAmountOut(address token, bool isBuy, uint256 amountIn) public view returns (uint256) {
2     if (bonding.isGraduating(token) || bonding.isGraduated(token)) revert TokenGraduated();
3     ...
4 }
```

RESOLUTION

Alt Fun Team - Acknowledged.

LOW R1-L-06

RESOLVED

Events can emit misleading amounts.

CATEGORY Unexpected Behavior

LOCATION src/Zap.sol:259-263

REPO Round 1 - Main Review

DESCRIPTION

`buyInternal` emits `Buy` and `Referred` event with the user-supplied `usdcAmount`, even when `_executeBuy` consumes only a small portion and refunds the rest. This happens on capped curve buys near graduation when `previewLtUntilGraduation` limits how much LT the curve still needs, so Zap may convert only the minimum required USDC, execute the capped buy, then return unused USDC and refunded fee portion to the trader.

TEXT

```
1      emit Buy(tokenAddress, msg.sender, usdcAmount, tokensOut);
2
3      if (referrer != address(0) && referrer != msg.sender) {
4          emit Referred(tokenAddress, msg.sender, referrer, usdcAmount);
5      }
```

A trader can submit a large buy when remaining graduation capacity is tiny, receive almost all USDC back in the same transaction, and function emits inflated `Buy` and `Referred` amounts, which may be unexpected for external listeners that treat these event amounts as executed trade volume.

RECOMMENDATION

Document that the mentioned fields represent the submitted gross amount, not the executed or non-refunded trade amount, or consider emitting the actual gross amount spent.

RESOLUTION

Alt Fun Team - Resolved in `alt-fun#1187`.

LOW R1-L-07

RESOLVED

`MIN\_USDC\_AMOUNT` can desync from LT floor.

CATEGORY Configuration

LOCATION src/Zap.sol:44

REPO Round 1 - Main Review

DESCRIPTION

Zap hardcodes `MIN_USDC_AMOUNT = 10e6` and uses it as a pre-flight floor on buys and sells so users see `BelowMinAmount()` instead of LT's undecodable `0x05eb05ac` (`BelowMinTransactionSize`). The matching value on the LT side is `GlobalStorage.minTransactionSize`, which is owner-settable and capped at `100e6` (`$100`) by `_MAX_MIN_TRANSACTION_SIZE_UNSCALED = 100`. Today both are `10e6` on mainnet, but they can legitimately raise their floor up to 10 Zap's constant without any on-chain notification to Alt Fun.

If that ever happens, trades passing Zap's check but below the new LT floor revert with the raw selector, only fix is a Zap UUPS upgrade.

RECOMMENDATION

Consider reading `minTransactionSize` live from `IBounceGlobalStorage` instead of hardcoding, or add an `onlyOwner` setter for `MIN_USDC_AMOUNT`.

RESOLUTION

Alt Fun Team - Resolved in `alt-fun#1186`.

LOW R1-L-08

RESOLVED

Graduation event overstates LP tokens.

CATEGORY Unexpected Behavior

LOCATION src/Bonding.sol:963

REPO Round 1 - Main Review

DESCRIPTION

During phase-1 graduation, `tokensForLP` and `ltFromPair` are stored as the intended amounts for LP seeding. During phase-2 finalization, an empty V2 pair receives exactly those stored amounts. However, if the V2 pair was already minted with non-empty reserves, finalization enters the rebalance path, may swap part of the stored amounts, and then calls `addLiquidity`, which only pulls the optimal balanced subset of the remaining balances. However, `TokenGraduated` is emitted with the cached phase-1 `tokensForLP` value rather than the actual token amount deposited by the router. In an imbalanced preseed scenario, the actual token amount deposited and locked can be lower or higher than `tokensForLP`. Off-chain listeners that treat `tokensInLP` as the actual locked token amount may overstate post-graduation liquidity and token backing.

RECOMMENDATION

Document that `tokensInLP` represents the cached phase-1 target amount, not the actual token amount deposited into the V2 LP, or consider emitting the actual token amount locked.

RESOLUTION

Alt Fun Team - Resolved in `alt-fun#1185`.

LOW **R1-L-09**

ACKNOWLEDGED

Floor-bump buys strand LT dust.

CATEGORY Logical Error

LOCATION src/Zap.sol:336-370

REPO Round 1 - Main Review

DESCRIPTION

Near graduation, a cap-binding buy may require less LT than BounceTech's minimum mint size. To avoid bricking tiny closing buys, `Zap` intentionally mints the minimum USDC amount and refunds unused LT directly to the buyer.

However, that refunded LT can be below the LT redeem floor, so the buyer cannot independently redeem it back to USDC. Even when the buyer sends the exact remaining USDC needed, decimal conversion between 6-decimal USDC and 18-decimal LT can leave a small LT remainder.

Therefore, the user entered a USDC-to-token flow but exits with unwanted leveraged-token dust exposure. Recovering the value may require acquiring more LT to cross the redeem floor and later paying the LT redemption fee.

RECOMMENDATION

Do not blindly remove the floor-bump, because it is needed to let tiny closing buys graduate. Instead, add an explicit `acceptLtDust` option or surface the expected LT refund before execution.

If `acceptLtDust == false` and the expected refund is below the redeem floor, revert with a clear custom error before minting.

RESOLUTION

Alt Fun Team - Acknowledged.

LOW R1-L-10

RESOLVED

`getAmountOut` overstates buys near sellout.

CATEGORY Logical Error

LOCATION src/Router.sol:45-67

REPO Round 1 - Main Review

DESCRIPTION

`Router.getAmountOut` (buy path) returns the raw curve output with no cap:

TEXT

```
1  if (isBuy) {
2      uint256 newReserveAsset = reserveAsset + amountIn;
3      uint256 newReserveToken = k / newReserveAsset;
4      return reserveToken - newReserveToken;    // uncapped
5  }
```

The real buy (`_computeBuy` , used by `buy()` and `previewBuy()`) clamps the output to the pair's actual token balance and back-solves the input:

TEXT

```
1  uint256 realBalance = pair.tokenBalance();
2  if (tokensOut > realBalance) {
3      tokensOut = realBalance;
4      uint256 cappedReserveToken = reserveToken - tokensOut;
5      uint256 cappedReserveAsset = (k + cappedReserveToken - 1) / cappedReserveToken;
6      amountInUsed = cappedReserveAsset - reserveAsset;
7  }
```

`reserveToken` is virtual (runs to the full 1B supply) and stays ~250M above the real sellable balance for the whole curve, so near sellout the two functions disagree.

RECOMMENDATION

Cap `getAmountOut` 's buy branch at `tokenBalance()` (or delegate to `_computeBuy`) so the quote matches execution, or document that `getAmountOut` is the raw curve quote and `previewBuy` is the one to use for buys.

RESOLUTION

Alt Fun Team - Resolved in `alt-fun#1184` .

INFO R1-I-01

RESOLVED

Inconsistent `balanceOf` vs `tokenBalance`.

CATEGORY Best Practices

LOCATION src/Bonding.sol:1009

REPO Round 1 - Main Review

DESCRIPTION

In `Bonding._prepareGraduationLiquidity`, the unsold-token snapshot reads the pair's launched-token balance directly via the ERC-20 interface:

TEXT

```
1    unsoldBurned = IERC20(tokenAddress).balanceOf(pairAddr);
```

Every other call site in `Bonding.sol` that needs the same value uses the dedicated helper exposed by `Pair`:

TEXT

```
1    // Pair.sol
2    function tokenBalance() external view returns (uint256) {
3        return IERC20(launchedToken).balanceOf(address(this));
4    }
```

Using `IPair(pairAddr).tokenBalance()` here would match the convention used in `canGraduate`, `_previewLtUntilGraduation`, and the rest of the file.

RECOMMENDATION

Consider replacing the direct ERC-20 call with the `Pair` helper for consistency:

TEXT

```
1    unsoldBurned = IPair(pairAddr).tokenBalance();
```

RESOLUTION

Alt Fun Team - Resolved in `alt-fun#1183`.

INFO R1-I-02

RESOLVED

`Usd` constants are actually USDC-denominated.

CATEGORY Documentation

LOCATION src/interfaces/IBounceLeveragedToken.sol:28

REPO Round 1 - Main Review

DESCRIPTION

Throughout `Bonding.sol`, several constants and storage slots are named with a `Usd` suffix (e.g. `VIRTUAL_LIQUIDITY_USD`, `graduationThresholdUsd`, and the local `valueUsd` in `canGraduate`). The naming implies these values are denominated in U.S. dollars. In reality, the protocol has no USD price oracle. The `exchangeRate()` of the BounceTech LT is defined as:

TEXT

```
1 // LeveragedToken.sol
2 return totalAssets().scaleFrom(_baseAsset().decimals()).div(totalSupply());
```

`totalAssets()` is the LT contract's raw USDC balance, and the result is USDC-per-LT scaled to 18 decimals. All Alt Fun arithmetic that compares against `*Usd` constants is therefore comparing USDC amounts (18-dp scaled), not USD amounts.

TEXT

```
1 /// @notice USD per LT unit, 18-dp.
2 function exchangeRate() external view returns (uint256);
```

RECOMMENDATION

Consider clearly documenting that all denominations are in USDC.

RESOLUTION

Alt Fun Team - Resolved in `alt-fun#1182`.

INFO **R1-I-03**

RESOLVED

Seed Fee Reduces Curve Floor.

CATEGORY Validation

LOCATION src/Zap.sol:223

REPO Round 1 - Main Review

DESCRIPTION

`Zap.createToken` enforces the mandatory seed minimum against gross USDC input, but `Zap._executeBuy` deducts the buy fee before minting LT and sending value into the bonding curve.

For example, at a 75 bps buy fee, an exact `20 USDC` seed passes the seed floor but only `19.85 USDC` reaches the LT/curve.

Therefore, launches can satisfy the advertised seed floor while providing slightly less net curve liquidity than the anti-snipe design implies.

RECOMMENDATION

If the intended floor is net curve liquidity, require `seedUsdcAmount - fee ≥ MIN_SEED_USDC` or exempt the mandatory seed buy from protocol fees. If the intended floor is gross user spend, update docs and UI copy to say the seed floor is before fees.

RESOLUTION

Alt Fun Team - Resolved in `alt-fun#1178`.

INFO R1-I-04

RESOLVED

Dust Fee Rounding Under-accrues.

CATEGORY Rounding

LOCATION src/Zap.sol:379

REPO Round 1 - Main Review

DESCRIPTION

For cap-binding buys, `Zap._executeBuy` computes:

TEXT

```
1   effectiveBaseSpent = (amountInUsed * baseToConvert) / ltMinted;
2   actualFee = (usdcAmount * buyFeeBps_ * effectiveBaseSpent) / (BPS_DENOM * netUsdc);
```

The intermediate `effectiveBaseSpent` division rounds down before the fee multiplication. The sell path has the same rounding direction at the direct fee calculation:

TEXT

```
1   fee = (grossUsdc * sellFeeBps) / BPS_DENOM;
```

Because the fee is owed to the protocol and creator, trader favorable rounding returns dust that should instead accrue to `FeeVault` under a protocol-favorable rounding policy.

RECOMMENDATION

Use a single full-precision expression or `Math.mulDiv` and round trade fees up in favor of the protocol and creator. For cap-binding buys, cap the rounded fee at `feeOnGross` so refunds cannot underflow and full-size buys never charge more than the fee already withheld.

For sells, compute the fee with upward rounding, for example `Math.mulDiv(grossUsdc, sellFeeBps, BPS_DENOM, Math.Rounding.Ceil)`. If keeping the current behavior, document the trader-favorable dust rounding as intentional.

RESOLUTION

Alt Fun Team - Resolved in `alt-fun#1180`.

INFO R1-I-05

RESOLVED

Zero LP Lock Can Be Rewritten.

CATEGORY Validation

LOCATION src/LPLock.sol:76

REPO Round 1 - Main Review

DESCRIPTION

`LPLock.recordLock` uses `locks[token].amount  $\neq$  0` as the only one-shot guard. A zero-amount lock stores `lpPair` and `lockedAt`, but leaves `amount` equal to zero, so a later allowlisted `recordLock` call for the same token can replace the recorded pair and amount instead of reverting as already locked.

RECOMMENDATION

Reject `amount == 0` in `LPLock.recordLock`.

RESOLUTION

Alt Fun Team - Resolved in `alt-fun#1179`.

INFO R1-I-06

ACKNOWLEDGED

Trader-Favorable Rounding in Curve Quote Math.

CATEGORY Rounding

LOCATION src/Router.sol:136

REPO Round 1 - Main Review

DESCRIPTION

`Router._computeBuy` and `Router._computeSell` both derive output amounts using expressions of the form `reserve - (k / newReserve)`. Because Solidity integer division rounds down, both `(k / newReserveAsset)` in the buy path and `(k / newReserveToken)` in the sell path round down. As a result, the surrounding subtraction rounds up, meaning `tokensOut` and `assetOut` are each rounded up in the trader's favor by up to 1 unit in the normal path.

Noting that this does not appear to violate the pair's intended invariant, because `Pair.swap` explicitly tolerates a 1-unit rounding shortfall by checking `(newTokenReserve + 1) * (newAssetReserve + 1) ≥ k` instead of enforcing the strict product directly.

RECOMMENDATION

If stricter accounting is preferred, round the post-trade reserve up so the resulting output value rounds down instead of up.

RESOLUTION

Alt Fun Team - Acknowledged.

INFO R1-I-07

RESOLVED

Zap stores an unused UniswapV2 router.

CATEGORY Best Practices

LOCATION src/Zap.sol:70

REPO Round 1 - Main Review

DESCRIPTION

Zap takes `uniswapV2Router` in initialize, requires it non-zero, stores it, and has a getter for it. Nothing ever calls it.

TEXT

```
1    \$.uniswapV2Router = IUniswapV2Router02(uniswapV2Router_);
```

Post-graduation swaps don't use the router. `_swap0nUniswapV2` pulls the pair from `bonding.graduatedPair(...)` and calls `pair.swap` directly, because HyperSwap's router has no canonical swap functions (see the comment on `_swap0nUniswapV2`). So the stored router never gets touched.

The storage comment claims it's a meaningful immutable ("migrating to a different HyperSwap fork requires a UUPS upgrade"), but nothing reads it on-chain, so that's not true. It just forces deployers to pass a live address that does nothing and burns an SSTORE.

Bonding's `uniswapV2Router` is a different story, it's actually used in `_routerDepositAndDispose` for `addLiquidity`. This is only about Zap's copy.

RECOMMENDATION

Drop the field, the initialize param, the zero-check, and the getter.

RESOLUTION

Alt Fun Team - Resolved in `alt-fun#1177`.

INFO R1-I-08

RESOLVED

Hardcoded 0.3% V2 fee desyncs from HyperSwap.

CATEGORY Logical Error

LOCATION src/Zap.sol:526

REPO Round 1 - Main Review

DESCRIPTION

Both swap paths that hit the HyperSwap V2 pair compute the output with a hardcoded 997/1000 fee numerator.

`Zap._swapOnUniswapV2` :

TEXT

```
1    uint256 amountInWithFee = amountIn * 997;
2    amountOut = (amountInWithFee * reserve0Out) / (reserveIn * 1000 + amountInWithFee);
```

and `Bonding._pairRebalance` :

TEXT

```
1    uint256 amountInWithFee = s * 997;
2    uint256 expectedOut = (amountInWithFee * p.reserve0Out) / (p.reserveIn * 1000 + amountInWithFee);
```

Neither reads the fee from the pair. The output is then handed straight to `pair.swap`, whose K-check uses the pair's real fee. The 0.3% assumption holds only as long as HyperSwap's fee stays at the canonical Uniswap V2 value.

RECOMMENDATION

Don't hardcode the fee. Read it from the pair and derive the numerator at call time, or quote through the pair's own fee-aware view if exposed. Or consider documenting the hard dependency on HyperSwap keeping the 0.3% fee.

RESOLUTION

Alt Fun Team - Resolved in `alt-fun#1181`.

INFO R1-I-09

RESOLVED

Documentation drift.

CATEGORY Documentation

LOCATION Global

REPO Round 1 - Main Review

DESCRIPTION

- **Opening market cap:** docs say approximately \$4,000 ; current source pins it to `Bonding.VIRTUAL_LIQUIDITY_USD = 3000e18` , so it is approximately \$3,000 .
- **Graduation threshold:** docs say \$12,000 / \$12K ; current `Bonding.graduationThresholdUsd()` is 9000e18 , so it is \$9,000 .
- **Graduation market cap:** docs say approximately \$16,000 ; current launch-rate value is `VIRTUAL_LIQUIDITY_USD + graduationThresholdUsd = $3,000 + $9,000` , so approximately \$12,000 .
- **Buy / sell fee:** docs say 0.5% per side; current `Zap.buyFeeBps()` and `Zap.sellFeeBps()` are 75 , so fees are 0.75% per side.
- **Fee split:** docs say 0.4% protocol / 0.1% creator; current `Zap.creatorFeeBps()` is 3333 , so the effective split is approximately 0.50% protocol / 0.25% creator.

RECOMMENDATION

Update the documentation to match the deployed contracts

RESOLUTION

Alt Fun Team - Resolved in alt-fun#1176 .

INFO R1-I-10

RESOLVED

Seed Buy Can Be Unwound.

CATEGORY Documentation

LOCATION src/Zap.sol:46

REPO Round 1 - Main Review

DESCRIPTION

`Zap.createToken` requires a creator seed buy of at least `MIN_SEED_USDC`. Comments and docs describe this seed as absorbing the cheap bottom of the curve before public trading opens.

However, the launch-delay gate only blocks buys. The creator can sell the seed tokens back into the curve before public trading opens.

RECOMMENDATION

Document this behavior, or block creator sells until public trading opens if the seed is meant to remain in the curve.

RESOLUTION

Alt Fun Team - Resolved in `alt-fun#1175`.

MEDIUM R2-M-01

RESOLVED

Tiny Minted Preseeds Still Cause Graduation Skew.

CATEGORY Logical Error

LOCATION src/Bonding.sol:1389-1400

REPO Round 2 - Main Remediation Review

DESCRIPTION

The remediation of H-01 substantially reduces the original attack and prevents the large skew previously observed at graduation. However, a small residual profit opportunity remains in the mint-preseed path of `finalizeGraduation`.

In the minted-preseed branch, `finalizeGraduation` enters `_seedRebalancing` and only falls back to `_seedDirectMint` when no rebalance swap can execute or when the quoted output is zero.

However, if the `expectedOut == 1` in `_pairRebalance`, the code treats the rebalance as good enough even when the swap is still too coarse to bring the pool close to the cached curve-close ratio.

In the replayed fuzzing reproducer, the hostile preseed is `141,638 TOKEN / 9 LT`, the rebalance swap returns only 1 LT, and the pool remains meaningfully LT-richer than intended. The subsequent router deposit then adds liquidity at that post-swap ratio, preserving the skew instead of restoring the cached graduation ratio.

While the resulting profit is far smaller than in the original issue, this still gives the attacker a better-than-fair post-graduation exit.

RECOMMENDATION

Consider checking the reserve ratios even after the rebalance and only allow some expected amount of skew (e.g. 0.5%) and fallback into the `_seedDirectMint` path otherwise.

Alternatively, widen the `_seedDirectMint` fallback requirements preemptively for tiny seeds/low reserves situations without any after-rebalance ratio checks.

RESOLUTION

Alt Fun Team - Resolved in `alt-fun#1208`.

INFO R2-I-01

RESOLVED

Unused Zap import.

CATEGORY Superfluous Code LOCATION Zap.sol:14 REPO Round 2 - Main Remediation Review

DESCRIPTION

Zap.sol imports IBounceGlobalStorage at src/Zap.sol:14 , but Zap never references that identifier after the import. Forge lint reports the same unused-import note for this line. This has no direct runtime fund-loss impact, but it leaves a stale BounceTech dependency in the user-facing router source and can mislead reviewers or integrators into thinking Zap reads BounceTech global storage directly

RECOMMENDATION

Remove the unused IBounceGlobalStorage import from Zap.sol . If Zap is intended to validate BounceTech global storage directly in a future version, wire that dependency explicitly and document the behavior instead of leaving the stale import.

RESOLUTION

Alt Fun Team - Resolved in alt-fun#1202 .

INFO R2-I-02

RESOLVED

Zap Bonding rotation lacks runbook.

CATEGORY Documentation

LOCATION Zap.sol:585

REPO Round 2 - Main Remediation Review

DESCRIPTION

`Zap.initialize` already sets the Bonding dependency during deployment, but `Zap.setBonding` can replace that dependency later at `src/Zap.sol:585`. The setter only validates that the new Bonding recognizes this Zap as a router; it does not verify that tokens launched through the previous Bonding exist in the new Bonding state. Once the pointer is changed, `Zap.buy` and `Zap.sell` resolve token ownership and lifecycle through the new Bonding, so old tokens that only exist in the previous Bonding are treated as non-trading and user flows revert.

The documentation describes the safer hot-swap direction as adding a new Zap to the Bonding allowlist, flipping the frontend, and removing the old Zap while the old Zap can continue serving the old Bonding state. There is no NatSpec or runbook explaining when reusing one Zap across a fresh Bonding proxy is safe, how old tokens remain tradable, or whether the function is only a leftover deployment hook.

RECOMMENDATION

If Bonding rotation is not required after initialization, remove `setBonding` and rely on deploying a new Zap for a new Bonding instance. If it is required, add explicit NatSpec and an operational runbook stating that the old Zap must remain available for tokens stored in the old Bonding, or define and enforce a state-migration/empty-state precondition before changing the pointer.

RESOLUTION

Alt Fun Team - Resolved in `alt-fun#1207`.

INFO R2-I-03

RESOLVED

Stale `MIN\_SEED\_USDC` Floor Docs.

CATEGORY Documentation

LOCATION src/Zap.sol:43

REPO Round 2 - Main Remediation Review

DESCRIPTION

`Zap.createToken` enforces `minSeedUsdc()`, not the bare `MIN_SEED_USDC` constant. `minSeedUsdc()` returns `max(MIN_SEED_USDC, grossed-up live mint floor)`, so the real floor exceeds `$20` whenever BounceTech's `minTransactionSize()` grossed up for the buy fee is larger. Two comments still call `$20` the floor:

- `MIN_SEED_USDC` natspec ("Mandatory seed-buy floor ... `$20`", rewritten in this diff but still omitting the mint-floor raise).
- `BelowMinSeed` natspec ("your launch seed must be at least `$20`").

RECOMMENDATION

Update both comments to state the enforced floor is `minSeedUsdc() = max(MIN_SEED_USDC, live mint floor grossed up for the buy fee)`, and point them at `minSeedUsdc()` like the call site already does.

RESOLUTION

Alt Fun Team - Resolved in `alt-fun#1205`.

INFO R2-I-04

RESOLVED

GlobalStorage rotation affects trades.

CATEGORY Documentation

LOCATION src/Bonding.sol:799-801

REPO Round 2 - Main Remediation Review

DESCRIPTION

`setBounceGlobalStorage` is documented as affecting future launches only, but live Zap trade checks read the current `GlobalStorage` through `Bonding`.

TEXT

```
1      function minUsdcAmount() public view returns (uint256) {
2          return _s().bonding.bounceGlobalStorage().minTransactionSize();
3      }
```

As a result, rotating `BounceTech GlobalStorage` can immediately change `minUsdcAmount` for already launched tokens, affecting buys, sells, and seed-size calculations, which is not in accordance with the `setBounceGlobalStorage NatSpec` stating that the change affects future launches only.

RECOMMENDATION

Update the `NatSpec` to state that `GlobalStorage` rotation also affects the live minimum USDC trade amount.

RESOLUTION

Alt Fun Team - Resolved in `alt-fun#1203`.

INFO R2-I-05

RESOLVED

`Router.initialize` missing factory zero-check.

CATEGORY Informational

LOCATION src/Router.sol:31-37

REPO Round 2 - Main Remediation Review

DESCRIPTION

`Router.initialize` stores the factory dependency with only a bare assignment and no validation:

TEXT

```
1 function initialize(address factory_) external initializer {
2     __AccessControl_init();
3     _grantRole(DEFAULT_ADMIN_ROLE, msg.sender);
4     factory = Factory(factory_); // no zero-address check
5 }
```

The contract already declares `error ZeroAddress();` at `Router.sol`, but it is never reverted anywhere in the contract, the author clearly intended a guard here that was never wired up.

RECOMMENDATION

Add the guard using the already-declared error, mirroring the other initializers.

RESOLUTION

Alt Fun Team - Resolved in `alt-fun#1204`.

INFO R2-I-06

RESOLVED

FeeVault rotation fragments claims.

CATEGORY Documentation

LOCATION Zap.sol:596-608

REPO Round 2 - Main Remediation Review

DESCRIPTION

`Zap._accrueFee` forwards each trade fee to the currently configured FeeVault and records the creator/protocol claim there. `Zap.setFeeVault` can later replace that configured vault at `src/Zap.sol:600`, but the setter only checks that the new vault allowlists Zap as a depositor. Existing creator and protocol balances are not migrated, because those balances are stored inside the old FeeVault instance. After such a rotation, new fees accrue in the new vault while old claimable balances remain in the previous vault.

This does not by itself burn funds: creators and the protocol can still claim from the old vault if it remains available. The issue is that the docs describe a single pooled FeeVault claim surface and router/Zap swapability with balances untouched, but they do not explain that replacing the FeeVault itself creates multiple claim locations or requires an explicit migration/dual-claim runbook.

RECOMMENDATION

If `FeeVault` replacement is not needed, remove `setFeeVault` and rotate Zaps/depositors within the same vault. If it is needed, document the operational runbook: keep the old vault discoverable and claimable, expose both old and new claim balances in the UI/API, or provide a migration path before redirecting future accruals.

RESOLUTION

Alt Fun Team - Resolved in `alt-fun#1206`.

INFO R2-I-07

ACKNOWLEDGED

Renounced ownership can break admin paths.

CATEGORY Informational

LOCATION Global

REPO Round 2 - Main Remediation Review

DESCRIPTION

Ownable contracts inherit `renounceOwnership` while still relying on owner for live administration and recovery. If `renounceOwnership` is called accidentally during a mistaken ownership operation, all `onlyOwner` functions become unavailable, including configuration updates and upgrades. In Bonding, this can also affect graduation finalization because LT residue is swept to the current owner.

RECOMMENDATION

Consider overriding `renounceOwnership` in ownable contracts to prevent accidental ownership renouncement.

RESOLUTION

Alt Fun Team - Acknowledged.

INFO R2-I-08

ACKNOWLEDGED

Single-step creator transfer.

CATEGORY Informational

LOCATION src/Bonding.sol:727-737

REPO Round 2 - Main Remediation Review

DESCRIPTION

`transferCreator` immediately replaces the creator without requiring the new address to accept. Future creator fees are credited to the creator address recorded at accrual time, and `FeeVault.claim` later pays only that credited address. If the creator is changed to a typo, inaccessible wallet, or contract that cannot call claim, future creator fees and creator-level control can become inaccessible.

RECOMMENDATION

Consider using a two-step creator transfer where the new creator must accept before the creator address is updated.

RESOLUTION

Alt Fun Team - Acknowledged.

INFO R2-I-09

RESOLVED

Factory router can be frozen wrong.

CATEGORY Validation

LOCATION src/Factory.sol:62-70

REPO Round 2 - Main Remediation Review

DESCRIPTION

Factory only checks that the router is non-zero before storing it, then permanently freezes the value after the first pair is created. New pairs store the factory router as their immutable authorized router.

If the factory is configured with a router that does not match the router used by Bonding, or with a router pointing to a different factory, the first successful pair creation can lock the system into an inconsistent configuration.

RECOMMENDATION

In `setRouter`, validate that the provided router points back to the factory before the first pair is created.

TEXT

```
1    if (address(Router(router_).factory()) != address(this)) revert InvalidRouter();
```

RESOLUTION

Alt Fun Team - Resolved in `alt-fun#1210`.

INFO R2-I-10

RESOLVED

`FeeVault.setFeeTo` redirects accrued fees.

CATEGORY Best Practices

LOCATION src/FeeVault.sol:179-187

REPO Round 2 - Main Remediation Review

DESCRIPTION

`FeeVault` accrues protocol fees into a single pooled `protocolBalance`, and `claimProtocol()` pays the **current** `feeTo` at claim time:

TEXT

```
1 function claimProtocol() external nonReentrant returns (uint256 amount) {
2     amount = \$.protocolBalance;
3     if (amount == 0) revert NothingToClaim();
4     \$.protocolBalance = 0;
5     address feeTo_ = \$.feeTo;          // ← read live, at claim time
6     \$.usdc.safeTransfer(feeTo_, amount);
7     ...
8 }
```

`setFeeTo` (`FeeVault.sol:179`) replaces `feeTo` with only a non-zero check and emits `FeeToUpdated`; it does not flush or migrate the outstanding `protocolBalance` first. As a result, if protocol fees accrue while `feeTo == A` and the owner later calls `setFeeTo(B)` before anyone calls `claimProtocol`, the entire accumulated `protocolBalance`, including the portion earned while `A` was the recipient, is paid to `B`. The same applies to `sweepDonations()`, which also pays the live `feeTo`.

RECOMMENDATION

Either (a) document on `setFeeTo` that rotation redirects the entire outstanding `protocolBalance` (and donations) to the new recipient and that `claimProtocol()` should be called before rotating; or (b) auto-settle the pending `protocolBalance` to the old `feeTo` inside `setFeeTo` before updating it, mirroring the rotation-safe, per-address handling of creator balances.

RESOLUTION

Alt Fun Team - Resolved in `alt-fun#1209`.

MEDIUM R3-M-01

RESOLVED

Pre-Seed LP Subsidy via Protocol LT Correction.

CATEGORY Logical Error

LOCATION Bonding.sol

REPO Round 3 - Remediation Review 2

DESCRIPTION

`Bonding._seedRebalancing` checks whether both sides of a pre-existing V2 pair reserve are below `DIRECT_MINT_PRESEED_BPS` (50 bps = 0.5%) of the graduation targets. If both pass, the code falls into `_seedDirectMint`, which transfers a portion of the protocol's own LT budget to the pre-seeded pair and calls `pair.sync()` before the graduation mint, equalising the reserve ratio.

However, the correction transfer itself grants the attacker indirect access to protocol LT. An attacker who preseeds with `preseedToken = 0.5% of tokensForLP` and `preseedLT = 1 wei` passes both sides of the band check. The corrected pair's LT side is then funded with approximately `0.5% of ltFromPair` from the protocol's budget. The attacker's pre-existing LP therefore represents roughly 0.5% of the post-graduation pool, backed in part by LT they never contributed.

At graduation the attacker holds ~49 bps of total LP. Redeeming it yields approximately 45 LT (\$45 at 1 USDC/LT). Their real cost is the USDC paid to acquire the preseed tokens on the bonding curve, which is recovered in full at LP redemption since the pool opens at the correct ratio. Net gain: ~45 LT per graduation event, funded entirely by the protocol's correction budget.

RECOMMENDATION

Consider reducing `DIRECT_MINT_PRESEED_BPS` substantially.

The originally suggested 0.5% tolerance was appropriate for an after-rebalance deviation check, not for a preemptive direct-mint cutoff. If this path is retained, the threshold should be small enough that any attacker LP minted before graduation remains economically negligible.

RESOLUTION

Alt Fun Team - Resolved in `alt-fun#1214`.

MEDIUM R3-M-02

ACKNOWLEDGED

Hostile Preseed Can Consume Swap Budget.

CATEGORY Logical Error

LOCATION src/Bonding.sol:1346-1349

REPO Round 3 - Remediation Review 2

DESCRIPTION

The remediation of the previous mint-preseed issue addresses the tiny-seed / coarse-swap case that was reproduced in `test_replay_PRESEED`. In that scenario, the new preemptive `_seedDirectMint` branch prevents the router from depositing at a meaningfully skewed post-swap ratio.

However, because Uniswap is permissionless, attackers can choose arbitrary TOKEN/LT amounts and any reserve ratios when preseeding. As a result, they can construct a preseed that is outside the new small-seed direct-mint band, but still so far from the cached curve-close ratio that `_pairRebalance` is unable to correct it with the protocol's available inventory. In these cases, the rebalance can consume essentially the full swap budget while still leaving the pool materially off-curve.

In the replayed reproducer (`test_replay_10`), the hostile preseed is approximately 177,514 TOKEN / 983,483 LT, while the cached graduation target is approximately 131,689,568 TOKEN / 554.659 LT. This makes the pool extremely LT-rich relative to the intended curve-close ratio. `_pairRebalance` then consumes essentially the full TOKEN-side swap budget (~130.37M TOKEN) and successfully executes, but the resulting pool is still materially off-curve. Because the rebalance returned true, `finalizeGraduation` proceeds into `_routerDepositAndDispose`, which deposits at that still-skewed post-swap ratio rather than restoring the cached graduation ratio.

RECOMMENDATION

Consider validating whether the projected rebalance can actually bring the pool sufficiently close to the cached curve-close ratio before executing it. If the projected post-rebalance state would still remain materially off-curve, the contract should avoid proceeding into `_routerDepositAndDispose` at that skewed ratio.

One possible mitigation is to introduce a dedicated error for unrecoverable hostile preseed states, where neither the rebalance path nor the fallback path can restore an acceptable ratio with the protocol's available inventory. In such cases, the transaction can revert and require manual intervention by the team to resolve graduation safely.

However, this does introduce a liveness / DoS tradeoff, so the acceptable approach depends on protocol priorities: if price integrity at graduation is more important than unconditional liveness, a fail-closed design with explicit operator intervention may be preferable to allowing graduation to complete at a materially attacker-favorable ratio. If liveness is the top priority, consider evaluating both the rebalance path and the fallback path, and selecting the outcome that remains closest to the cached curve-close ratio in order to minimize attacker profit as much as possible.

RESOLUTION

Alt Fun Team - Acknowledged.

INFO R3-I-01

RESOLVED

Factory <-> Router Circular Import.

CATEGORY Best Practices LOCATION packages/contracts/src/Factory.sol:7

REPO Round 3 - Remediation Review 2

DESCRIPTION

`Factory.sol` imports the concrete `Router` contract to call `.factory()` inside the new `RouterFactoryMismatch` guard, while `Router.sol` already imports `Factory`. This creates a mutual import cycle between the two contracts.

RECOMMENDATION

Introduce a minimal `IRouter` interface in `Factory.sol` exposing only `factory()` external view returns (address), and use that instead of the import.

RESOLUTION

Alt Fun Team - Resolved in `alt-fun#1216`.

§ 04

Disclaimer.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

About Guardian.

Founded in 2022 by DeFi experts, Guardian is a leading audit firm in the DeFi smart contract space. With every audit report, Guardian upholds best-in-class security while achieving our mission to relentlessly secure DeFi.